

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Create Month column (YYYY-MM format)
df['Month'] = df['Date'].dt.to_period('M')

# Calculate total sales = Quantity * Price
df['Sales'] = df['Quantity'] * df['Price']

# Group by Month and calculate total sales
monthly_sales = df.groupby('Month')['Sales'].sum()

# Find the month with the highest total sales
best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()
print(f"Best month: {best_month}")
print(f"Total sales: ${highest_sales:.2f}")
```

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

product_sales=df.groupby('Product')['Quantity'].sum()
# Find the product with the highest total quantity sold
best_product =product_sales.idxmax()
highest_quantity = product_sales.max()

# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)

# write the code..
product_sales = df.groupby('City')['Quantity'].sum()
best_city = product_sales.idxmax()
# Display the result
print(f"City sold the most products: {best_city}")
```

```
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

grouped = df.groupby('Date')['Product'].apply(list)

# Store all product pairs
pair_counts = Counter()

for products in grouped:
    # Get all unique pairs for each date
    pairs = combinations(sorted(products), 2)
    pair_counts.update(pairs)

# Find the maximum frequency
max_count = max(pair_counts.values())

# Output the most frequent product pairs
for pair, count in pair_counts.items():
    if count == max_count:
        print(f"{pair[0]} and {pair[1]}: {count} times")
```

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# 1. Display the first 5 rows of the dataset
print(data.head())

# 2. Display the last 5 rows of the dataset
print(data.tail())

# 3. Get the shape of the dataset
print(data.shape)

# 4. Get a summary of the dataset (info)
print(data.info())

# 5. Get basic statistics of the dataset
print(data.describe())

# 6. Check for missing values
print(data.isnull().sum())

# 7. Fill missing values in the 'Age' column with the median age
data['Age'].fillna(data['Age'].median(), inplace=True)

# 8. Fill missing values in the 'Embarked' column with the mode
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)

# 9. Drop the 'Cabin' column due to many missing values
data.drop('Cabin', axis=1, inplace=True)

# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['IsAlone'] = data['FamilySize'].apply(lambda x: 1 if x == 0 else 0)

# 2. Convert 'Sex' to numeric
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode 'Embarked' (drop first category)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 4. Mean age
print(data['Age'].mean())

# 5. Median fare
print(data['Fare'].median())

# 6. Number of passengers by class
print(data['Pclass'].value_counts())

# 7. Number of passengers by gender
print(data['Sex'].value_counts())

# 8. Number of passengers by survival status
print(data['Survived'].value_counts())

# 9. Survival rate
print(data['Survived'].mean())

# 10. Survival rate by gender
print(data.groupby('Sex')['Survived'].mean())
```